

PHASE2_SERVICE_STACK

Phase-2 Backing-Service Stack — Design (HelixAgent → HelixLLM end-to-end)

Document	Phase-2 backing-service stack design (no code) — rootless-podman infra behind the HelixAgent → HelixLLM end-to-end flow
Revision	1 · Created 2026-07-06
Status	DESIGN READY — pending build-phase pin verification (§11.4.6 open items in §7)
Track/branch	(T1/main) → feature/helixllm-full-extension
Constitution HEAD followed	0882b9e (through §11.4.182)
Basis	04_implementation_plan.md P1-T2 / P2 / P3-T7 · RESUME.md live-state · submodules/containers (digital.vasic.containers) public API · submodules/helix_llm/docs/API_CONTRACT.md · HelixAgent .env.example + compose files (real, source-verified)

Scope. This document designs ONLY the Phase-2 *backing-service stack* — the rootless-podman infrastructure (PostgreSQL, Redis, Cognee, a vector DB, plus Cognee’s transitive graph store) that HelixAgent needs so a real generate request against the live HelixLLM persists a session, caches, and writes a memory. It does NOT design the LLM serving core (Phase 0/1, already proven live) nor the /v1 gateway (Phase 2 P2-T1). It is a design (no new code); Go snippets below are *usage/call shapes* of the existing digital.vasic.containers API, not new implementation.

0. What the repo already gives us (survey — §11.4.74 reuse-don't-reimplement)

The backing services are **already wired**, parameterised, and env-injected across three real compose files and HelixAgent's `.env.example`. Phase-2's job is to boot them **through the containers submodule's `pkg/boot/pkg/compose/pkg/health primitives`** (§11.4.76 on-demand-infra invariant) rather than `podman-compose up`, pin the images (the in-tree files use forbidden `:latest` tags — §11.4.99), and prove the end-to-end runtime signature (§4).

Source-verified evidence:

- **submodules/containers/pkg/compose/helix_project.go:89-312** — `DefaultHelixServices()` already ships `postgres-primary` (`postgres:16-alpine`), `redis-master-*` (`redis:7-alpine`) with real health checks (`pg_isready`, `redis-cli ping`).
 - **submodules/helix_agent/docker-compose.helixllm-infra.yml** — HelixLLM app infra: `postgres:16-alpine + redis:7-alpine + qdrant/qdrant:latest (+ Kafka)`, all with `${VAR:-default}` env injection and `mem_limit/pids_limit/oom_score_adj` set (§12.3 hygiene).
 - **submodules/helix_agent/docker-compose.memory-infra.yml** — HelixMemory infra: `qdrant/qdrant:latest + neo4j:5-community + redis:7-alpine + pgvector/pgvector:pg16`.
 - **submodules/helix_agent/docker-compose.memory.yml** — Cognee (`ghcr.io/topoteretes/cognee:latest`) wired to `VECTOR_DB_PROVIDER=qdrant, GRAPH_DB_PROVIDER=neo4j, DATABASE_* → Postgres (+ mem0, letta siblings)`.
 - **submodules/helix_agent/.env.example** — the confirmed env-var contract HelixAgent reads (§3): `HELIX_LLM_ENDPOINT, HELIX_LLM_DB_*, HELIX_LLM_REDIS_*, HELIX_LLM_VECTOR_DB=qdrant, HELIX_MEMORY_COGNEE_ENDPOINT, HELIX_MEMORY_QDRANT_ENDPOINT, HELIX_MEMORY_NEO4J_*, HELIX_MEMORY_REDIS_*`.
 - **submodules/helix_agent/internal/database/cognee_memory_repository.go + internal/database/db.go:324-351** — HelixAgent persists agent memory in a Postgres table `cognee_memories(session_id, dataset_name, content, vector_id, graph_nodes, search_key, ...)`. `vector_id ↔ Qdrant point`, `graph_nodes ↔ Neo4j`. **This table is the concrete sink for the §4 runtime signature.**
-

1. Service inventory

Ports below are the host-side ports the in-tree compose files already use; all are > 1024, so **rootless podman binds them without privilege**. RAM figures are (EST – measure) per §11.4.6 (measure the real RSS at boot; do not ship the estimate as fact). None of these services touches the GPU — all are **CPU-only** (confirmed in submodules/helix_llm/docs/VRAM_BROKER.md:29: “CPU-only | Qdrant, HelixMemory, ... | no GPU reservation | 0 GB GPU”), so they never contend with the RTX 5090 serving fleet.

#	Service	Purpose (in this stack)	Recommended pinned image	Host port(s)	CPU bu (E me
1	PostgreSQL (session/app store)	HelixLLM session/app persistence — the durable record a generate call writes (HELIX_LLM_DB_*)	docker.io/pgvector/pgvector:pg17 (postgres 17 + pgvector superset; one image serves both roles)	5433→5432 (HelixLLM), 5434→5432 (memory)	~0. idl 512 (ES mer 4g alr
2	Redis (cache)	HelixLLM response/route cache + memory-service cache (HELIX_LLM_REDIS_*, HELIX_MEMORY_REDIS_*)	docker.io/redis:7-alpine (in-use pin; §7-Q2 re: redis 8)	6381→6379 (HelixLLM), 6380→6379 (memory)	~0. idl 128 (ES mer 1– set
3	Cognee (AI memory / knowledge-graph engine)	Ingest+query long-term agent memory; writes the vector + graph the §4 signature asserts (HELIX_MEMORY_COGNEE_ENDPOINT,/api/v1/health)	docker.io/cognee/cognee:1.2.2 (API server — not cognee-mcp; §7-Q4 bloat)	8000→8000	~1- un loa 2 C (ES mer 2– set
4	Qdrant (vector DB — chosen; see §1.1)	Vector store for Cognee/mem0/RAG embeddings; vector_id in cognee_memories resolves here (HELIX_MEMORY_QDRANT_ENDPOINT,/healthz)	docker.io/qdrant/qdrant pinned by digest	6333→6333 (HTTP), 6334→6334 (gRPC)	~0. idl 500 (ES

#	Service	Purpose (in this stack)	Recommended pinned image	Host port(s)	CP bu (E me
			(~v1.18.x; §7-Q3)		me 4g
5	Neo4j (graph store — <i>transitive</i> Cognee dep)	Cognee’s GRAPH_DB_PROVIDER=neo4j knowledge graph; graph_nodes in cognee_memories resolves here (HELIX_MEMORY_NEO4J_*)	docker.io/neo4j:5-community (§7-Q5 re: calendar versioning)	7474→7474 (HTTP), 7687→7687 (Bolt)	~1 un loa Gil (JV he me 4g

Why Neo4j is in the inventory though the brief named four services. The brief named PostgreSQL, Redis, Cognee, and a vector DB. Cognee is not self-contained: its own compose (docker-compose.memory.yml) hard-depends on a **graph** store (GRAPH_DB_PROVIDER=neo4j, depends_on: [postgres, qdrant, neo4j]). Omitting Neo4j would leave Cognee unable to write graph_nodes, so the §4 memory-write signature could not pass. It is recorded here honestly (§11.4.6) as a transitive dependency, not silently added or silently dropped.

1.1 Vector DB decision — Qdrant (not ChromaDB)

Decision: Qdrant. Rationale, cited:

1. **Reuse, not a new dependency (§11.4.74).** Qdrant is already the vector store in *all three* in-tree composes and both memory engines — Cognee VECTOR_DB_PROVIDER=qdrant (docker-compose.memory.yml), mem0 VECTOR_STORE=qdrant, and HelixAgent’s HELIX_LLM_VECTOR_DB=qdrant / HELIX_MEMORY_QDRANT_ENDPOINT (.env.example). Choosing ChromaDB would mean introducing a *new* backing service the catalogue does not use and rewiring Cognee/mem0 — the opposite of reuse.
2. **CPU-only, zero GPU contention.** VRAM_BROKER.md: 29 explicitly classes Qdrant CPU-only with a 0 GB GPU reservation — it never competes with the coder fleet on the RTX 5090.
3. **Production posture.** Qdrant is a single Rust binary exposing both REST (:6333) and gRPC (:6334), with payload filtering, quantization (TurboQuant/binary), and an unprivileged image variant — well-suited to rootless podman and concurrent multi-collection use. See Qdrant docs + releases (Sources, §8).

4. **ChromaDB trade-off (honest).** ChromaDB is an excellent embedded/Python-first store, but here it would be a second, less-hardened service for concurrent server workloads and would *not* be the store Cognee/mem0 speak to natively — so it adds integration surface for no reuse benefit. Qdrant wins on reuse + already-proven wiring, which is decisive under §11.4.74.

Honest boundary (§11.4.6): this is a reuse+fit decision for THIS repo’s existing wiring, not a universal “Qdrant beats Chroma” benchmark claim.

1.2 Rootless-podman notes (all five services)

- **No privilege / no GPU device.** Every image is CPU-only; none needs `--device nvidia.com/gpu` or `--privileged`. All host ports are > 1024 , so rootless podman maps them without `net.ipv4.ip_unprivileged_port_start` changes.
 - **Registries.** `docker.io` (postgres/pgvector, redis, qdrant, neo4j, cognee) pull fine rootless. Pin by **digest** at build time (`podman pull ... && podman image inspect --format '{{.Digest}}'`) so the tracked pin is reproducible (§11.4.99/§11.4.6) — the in-tree `:latest` tags (qdrant, cognee) are a drift hazard to fix.
 - **SELinux (ALT-Linux host).** Named-volume mounts may need `:Z/:z` relabel or `--security-opt label=disable` (the same flag Phase-0 GPU passthrough used). Prefer per-volume `:Z` over blanket `label=disable` for the data stores.
 - **Volumes.** Use the named volumes the composes already declare (`helixllm_postgres_data`, `helixllm_qdrant_data`, `neo4j_data`, `cognee_data`, ...); under rootless podman they live in the user’s `$XDG_DATA_HOME/containers/storage`. Data volumes are gitignored; regeneration is “re-boot the stack + re-ingest” (§11.4.77).
 - **Container-to-container DNS.** Cognee reaches Postgres/Qdrant/Neo4j by *service name* (`DATABASE_HOST=postgres`, `VECTOR_DB_URL=http://qdrant:6333`, `GRAPH_DB_URL=bolt://neo4j:7687`), which requires all five on **one shared podman network** (the composes define `helixmemory/helix`). HelixAgent (on the host) reaches them by `localhost:<hostport>`.
 - **Resource caps (§12.3/§12.6).** Keep the existing `mem_limit/memswap_limit/pids_limit/oom_score_adj` per service; the containers submodule’s `pkg/policy` is the Go counterpart if caps need to be applied programmatically.
-

2. How the stack boots via the containers submodule (§11.4.76)

Invariant: the stack is booted through `digital.vasic.containers` primitives, as part of the test/run entry point — never a hand-run `podman-compose up` (that bypasses the flow `CONST-030/§11.4.76` exist to prevent). All configuration is **env-injected** (§11.4.28); the submodule stays project-agnostic (the consumer supplies values).

2.1 Boot path (the `pkg/boot` → `pkg/health` shape)

The canonical shape (from `submodules/containers/README.md` Quick Start + `pkg/boot/options.go` + `pkg/health/checker.go`): auto-detect the rootless runtime, declare one `endpoint.ServiceEndpoint` per service (host, port, health type, compose file, service name, required), construct a `BootManager`, and `BootAll`. Health is a real TCP/HTTP probe (`pkg/health HealthType ∈ tcp|http|grpc|custom`), not a metadata check.

```
// USAGE SHAPE (not new code) – booting the Phase-2 stack via the
// containers submodule.
rt, _ := runtime.AutoDetect(ctx) // picks rootless
// podman on this host
endpoints := map[string]endpoint.ServiceEndpoint{
    "helixllm-postgres": endpoint.NewEndpoint().
        WithHost(env("HELIX_LLM_DB_HOST", "localhost")).WithPort(env("HELIX_LLM_DB_PORT", "5432")).
        WithHealthType("tcp").WithRequired(true).
        WithComposeFile("submodules/helix_agent/docker-compose.helixllm-infra.yml").
        WithServiceName("helixllm-postgres").Build(),
    "helixllm-redis": /* tcp :6381 ... WithServiceName("helixllm-redis") */,
    "qdrant": /* http :6333 /healthz ... WithServiceName("helixllm-qdrant") */,
    "neo4j": /* tcp :7687 ... memory-infra ... */,
    "cognee": /* http :8000 /api/v1/health ... WithRequired(true) */,
}
mgr := boot.NewBootManager(endpoints,
    boot.WithRuntime(rt),
    boot.WithHealthChecker(health.NewDefaultChecker()),
    boot.WithOrchestrator(composeOrch), // pkg/compose – compose up/down (local, rootless)
    boot.WithLogger(logging.NewSlogAdapter()),
    boot.WithProjectDir(repoRoot),
```

```

)
summary, err := mgr.BootAll(ctx) // starts +
    health-gates; required-fail ⇒ boot fails
// summary.Started / summary.Failed → captured to docs/qa/
    <run>/p2_boot_summary.txt

```

Alternative/complementary: `pkg/compose.NewHelixComposeProject(name, []HelixService{...})` (see `helix_project.go`) renders the stack as a typed `HelixComposeProject` with per-service `HelixHealthCheck` + `HelixResourceLimits`, so the compose definition itself can be generated from env-injected `HelixService` values instead of a checked-in YAML — the consumer builds the `[]HelixService` list (never hardcoded in the submodule, §11.4.28).

2.2 Health-gating (the on-demand-infra proof)

`mgr.BootAll` runs each endpoint’s health probe with retry (`pkg/health + pkg/health/retry.go`); a `Required:true` service that never goes healthy fails the boot (returns non-nil `err`). The real probes already exist in the compose files and map 1:1 to `HealthType`:

Service	Probe (compose healthcheck)	HealthType
postgres	<code>pg_isready -U ... -d ...</code>	<code>tcp:5433/5434</code> (or custom exec)
redis	<code>redis-cli ... ping</code>	<code>tcp:6381/6380</code>
qdrant	<code>GET /healthz</code>	<code>http:6333</code>
neo4j	<code>cypher-shell RETURN 1</code>	<code>tcp:7687</code> (or custom)
cognee	<code>GET /api/v1/health</code>	<code>http:8000</code>

This turns “did the stack come up?” into positive captured evidence (§11.4.5/ §11.4.69): the boot summary + each health verdict is written under `docs/qa/<run>/`.

2.3 On-demand + placement

Boot is lazy/on-demand (`pkg/lifecycle LazyBooter`) so the test entry point boots what it needs and idle-shuts-down after. All five run **local** on the RTX host (they are CPU-only companions to the GPU serving fleet), so remote distribution (`CONTAINERS_REMOTE_* / pkg/distribution`) is **not** engaged for this stack — but the same endpoints map would distribute unchanged if a future topology moves the memory tier to another host (§11.4.28).

3. HelixAgent → HelixLLM end-to-end wiring (env-injected, §11.4.28)

Every value below is a **real, source-verified** key from HelixAgent

`.env.example/configs/production.yaml` — not guessed (§11.4.6). Secrets stay in `.env` (gitignored, §11.4.10); nothing is hardcoded.

3.1 Pointing HelixAgent at the live HelixLLM

Env var	Value	Source / note
USE_HELIX_LLM	true	<code>.env.example</code>
HELIX_LLM_ENDPOINT	<code>http://localhost:8443</code> (default)	<code>.env.example</code> ; but see §7-Q1 — the <i>proven live</i> OpenAI-compatible server today is <code>http://localhost:18434/v1</code> (RESUME.md), and the Phase-2 gateway target is <code>:8100/v1</code> (04_implementation_plan.md P6-T1). Three candidate endpoints exist; the one HelixAgent uses for the Phase-2 end-to-end run must be chosen + pinned, not assumed.
HELIX_LLM_API_KEY	(from <code>.env</code>)	API-key auth guards the 7 gateway <code>/v1</code> LLM routes (API_CONTRACT.md §2); the raw llama-server on <code>:18434</code> is unauthenticated.
HELIX_LLM_TLS_SKIP_VERIFY	true (dev)	HelixLLM gateway on <code>:8443</code> is TLS-mandatory, TLS 1.3, HTTP/3+HTTP/2 (API_CONTRACT.md §1); the <code>:18434</code> llama-server is plain HTTP.

The HelixLLM endpoint HelixAgent calls is the OpenAI-compatible surface from API_CONTRACT.md §2: `POST /v1/chat/completions`, `POST /v1/completions`, `GET /v1/models`, `POST /v1/embeddings`, `POST /v1/messages` (Anthropic).

3.2 Pointing HelixAgent at each backing service

Concern	Env vars (confirmed)	Target
HelixLLM session/ app Postgres	HELIX_LLM_DB_HOST, HELIX_LLM_DB_PORT, HELIX_LLM_DB_NAME, HELIX_LLM_DB_USER, HELIX_LLM_DB_PASSWORD	service #1 (:5433)
HelixLLM cache Redis	HELIX_LLM_REDIS_HOST, HELIX_LLM_REDIS_PORT, HELIX_LLM_REDIS_PASSWORD	service #2 (:6381)
Vector store	HELIX_LLM_VECTOR_DB=qdrant, HELIX_LLM_QDRANT_HTTP_PORT, HELIX_LLM_QDRANT_GRPC_PORT; memory side HELIX_MEMORY_QDRANT_ENDPOINT=http:// localhost:6333	service #4
Cognee memory	HELIX_MEMORY_COGNEE_ENDPOINT=http:// localhost:8000, HELIX_MEMORY_COGNEE_API_KEY (empty ⇒ local container path)	service #3
Graph store	HELIX_MEMORY_NE04J_ENDPOINT=bolt:// localhost:7687, HELIX_MEMORY_NE04J_USER, HELIX_MEMORY_NE04J_PASSWORD	service #5
Memory-tier cache	HELIX_MEMORY_REDIS_ENDPOINT=localhost:6380, HELIX_MEMORY_REDIS_PASSWORD	service #2 (memory port)
Service discovery/ health (prod)	SVC_COGNEE_HOST/PORT/REMOTE/DISCOVERY_* (configs/production.yaml:321-328)	wires Cognee through the containers- submodule discovery/ health seam

3.3 The end-to-end path

HelixAgent (host binary)

```
| OpenAI-compatible request → HELIX_LLM_ENDPOINT (HelixLLM /v1/chat/
|                               | real inference on RTX 5090 (Pha
| session persist             → HELIX_LLM_DB_*           (Postgres #1)
| cache put/get               → HELIX_LLM_REDIS_*         (Redis #2)
```

```
└─ memory remember → HELIX_MEMORY_COGNEE_ENDPOINT (Cognee #3)
                      │
                      └─ vector upsert → Qdrant #4 (cogn
                        │
                        └─ graph write → Neo4j #5 (cogn
row lands in Postgres table `cognee_memories`
(internal/database/cognee_memory_repository.
```

4. §11.4.108 runtime signature — “Phase-2 end-to-end works”

Definition of done (single machine-checkable signature, verified on a clean boot):

One real HelixAgent generate request, driven through HelixAgent against the live HelixLLM, produces **all four** downstream side-effects, each asserted from the service’s own state (not from HelixAgent logs):

(S1) LLM — HelixLLM returns non-empty completion text for a fresh prompt AND `nvidia-smi` shows a VRAM delta during the call (real inference, not a stub). **(S2) Postgres session** — a new row exists in the HelixLLM session store *and* a new `cognee_memories` row (matching `session_id`) — proven by `psql ... SELECT count delta`, not by an HTTP 200. **(S3) Redis cache** — a new key attributable to the request exists — proven by `redis-cli ... DBSIZE/KEYS delta`. **(S4) Memory write** — the `cognee_memories` row’s `vector_id` resolves to a live **Qdrant** point (`GET /collections/<c>/points/<id> → found`) AND its `graph_nodes` resolve to a **Neo4j** node (`cypher-shell MATCH ... RETURN count>0`).

Captured evidence for every sub-assertion goes to `docs/qa/<run-id>/p2_e2e/:llm_response.json, nvidia_smi_before_after.txt, pg_session_delta.txt, pg_cognee_row.txt, redis_dbsize_delta.txt, qdrant_point.json, neo4j_node.txt`. A PASS with any sub-assertion missing/empty is a §11.4/§11.4.1 PASS-bluff (metadata-only), not a pass. Feature-class tags per §11.4.69: `boot_service` (§2 health) + `storage_write` (S2/S4) + the LLM liveness of S1.

4.1 RED-first polarity (§11.4.115) + standing guard (§11.4.135)

Author the check as a single RED_MODE-switched test: RED_MODE=1 runs it against the stack with Cognee/memory **disabled** (or Qdrant unreachable) and asserts the memory-write **absent** (reproduces the “generate works but nothing persisted” defect on the pre-fix artifact); RED_MODE=0 is the standing GREEN regression

guard asserting all four side-effects **present**. Register it in the Phase-8 regression suite.

4.2 Golden-good / golden-bad self-validation (§11.4.107(10))

The assertion harness (the analyzer that decides S1–S4 pass) is itself validated by a fixture pair so it provably cannot bluff:

- **golden-good fixture** — a captured bundle where the LLM text is non-empty, the Postgres/Qdrant/Neo4j deltas are all present → the analyzer **MUST** return PASS.
- **golden-bad fixture** — a bundle seeded with the exact failure the stack produces when memory is down: LLM text present but `cognee_memories` delta = 0, `vector_id` NULL / Qdrant point 404, Neo4j count = 0 → the analyzer **MUST** return FAIL.

Wire both into meta-test; a paired §1.1 mutation (e.g. weaken S4 to “row exists” without resolving the Qdrant point) **MUST** flip the golden-bad fixture to PASS → gate FAILs. An analyzer that passes its golden-bad fixture is the bluff this guards against.

5. Boot/verify sequence (operator + agent, at a glance)

1. `.env` present (gitignored, §11.4.10) with the §3 keys; images **digest-pinned** (§7).
 2. Agent boots the stack via `pkg/boot.BootManager.BootAll` (§2) — rootless podman, shared network, health-gated; capture `p2_boot_summary.txt`.
 3. Confirm each service healthy via `pkg/health` probes (§2.2) — captured.
 4. Point HelixAgent at HelixLLM + backing services via §3 env vars.
 5. Run the §4 signature (`RED_MODE=0`) on the clean boot; capture the 7 artefacts.
 6. GREEN only when S1–S4 all hold with captured evidence; else re-enter systematic debugging (§11.4.102) — never mark done on build-success (§11.4.108).
-

6. Composition footer

Composes / is bound by: §11.4.76 (containers-submodule on-demand infra — the boot path) · §11.4.161 (rootless container runtime) · §11.4.74 (reuse Qdrant/Cognee/ compose wiring, don’t reimplement) · §11.4.28 / **CONST-045** / **CONST-046** / **CONST-051** (env-injected, no hardcoded hosts/ports/content, submodule

decoupled) · §11.4.99 / §11.4.150 (latest-source pin verification; multi-angle image-tag research) · §11.4.108 (runtime-signature = done) · §11.4.107(10) (golden-good/bad self-validated analyzer) · §11.4.115 / §11.4.135 (RED-first polarity + standing regression guard) · §11.4.5 / §11.4.69 (captured sink-side evidence; boot_service/storage_write) · §11.4.83 (docs/qa/<run-id>/ evidence) · §11.4.6 (facts + honest open questions, §7) · §11.4.10 (secrets in .env) · §12.3 / §12.6 (per-container resource caps). Track (T1/main) on feature/helixllm-full-extension (§11.4.181 / §11.4.182).

7. Open questions / items to verify at build time (§11.4.6 — do not invent)

- **Q1 — Which HelixLLM endpoint does HelixAgent call for the Phase-2 run?** .env.example defaults HELIX_LLM_ENDPOINT=http://localhost:8443 (TLS gateway), the *proven live* server is http://localhost:18434/v1 (llama-server, plain HTTP), and P6-T1 names http://localhost:8100/v1 (planned router). Decide + pin one before the §4 run; today only :18434 is proven live (RESUME.md).
- **Q2 — Redis pin.** In-use is redis:7-alpine; upstream latest is Redis 8.x (redis:8-alpine). Recommend keeping 7-alpine (matches in-use, avoids drift) unless a Redis-8 feature is needed; either way pin a digest.
- **Q3 — Qdrant exact latest.** Web sources returned inconsistent dates (v1.17.1 in one, v1.18.2 in another with implausible “2024” dates). Do **not** ship a version string as fact — pin by `podman pull qdrant/qdrant && ...` Digest at build time and record the resolved digest.
- **Q4 — Cognee image + bloat.** Use the API-server image cognee/cognee:1.2.2 (latest stable, 2026-06-26), **not** cognee/cognee-mcp (Docker Hub reports ~27.8 GB, issue #3691 — CUDA/torch bloat) — the memory tier here is CPU-only, so the smaller API image is correct (§11.4.77 disk budget). The in-tree compose uses ghcr.io/topoteretes/cognee:latest (a :latest drift hazard) — replace with the digest-pinned cognee/cognee:1.2.2.
- **Q5 — Neo4j version.** In-use neo4j:5-community; Neo4j has since moved to calendar versioning (2025.xx). Keep 5-community unless Cognee requires newer; verify Cognee 1.2.2’s supported Neo4j range at build.
- **Q6 — cognee_integration is DISABLED in configs/development.yaml:317** — comment: “Disabled due to upstream Cognee bug (see COGNEE_BUG.md)”. The referenced COGNEE_BUG.md was **not found** in submodules/helix_agent/. This is a real blocker to the §4 memory-write signature: the upstream bug must be identified/re-verified against Cognee 1.2.2 (which may already fix it) before Phase-2 memory-write can pass. Track as a workable item; do not assume it is resolved.

- **Q7 — :latest tag removal.** Both `qdrant/qdrant:latest` and `ghcr.io/topoteretes/cognee:latest` in the in-tree composes violate §11.4.99 pin discipline; the Phase-2 compose/`HelixService` definitions must carry digest pins.
-

Sources verified 2026-07-06

- PostgreSQL / pgvector images: https://hub.docker.com/_/postgres · <https://hub.docker.com/r/pgvector/pgvector/tags> · <https://github.com/pgvector/pgvector>
- Redis image (8.x latest, 7-alpine in-use): https://hub.docker.com/_/redis · <https://github.com/redis/redis/releases>
- Qdrant releases (version to pin by digest — §7-Q3): <https://github.com/qdrant/qdrant/releases> · <https://hub.docker.com/r/qdrant/qdrant> · <https://qdrant.tech/documentation/installation/>
- Cognee releases (v1.2.2, 2026-06-26) + image bloat issue #3691: <https://github.com/topoteretes/cognee/releases> · <https://hub.docker.com/r/cognee/cognee> · <https://github.com/topoteretes/cognee/issues/3691>
- In-repo (source-verified, no URL): `submodules/containers/pkg/{boot,compose,health}` · `submodules/helix_llm/docs/API_CONTRACT.md` · `submodules/helix_llm/docs/VRAM_BROKER.md:29` · `submodules/helix_agent/{.env.example, docker-compose.helixllm-infra.yml, docker-compose.memory-infra.yml, docker-compose.memory.yml, configs/production.yaml, configs/development.yaml, internal/database/cognee_memory_repository.go, internal/database/db.go}` · `docs/research/07.2026/00_master/{04_implementation_plan.md, RESUME.md}`

Deep-research 2026-07-06: <https://github.com/qdrant/qdrant/releases> · <https://github.com/topoteretes/cognee/releases> · <https://github.com/topoteretes/cognee/issues/3691> · https://hub.docker.com/_/postgres · https://hub.docker.com/_/redis · <https://hub.docker.com/r/pgvector/pgvector/tags>